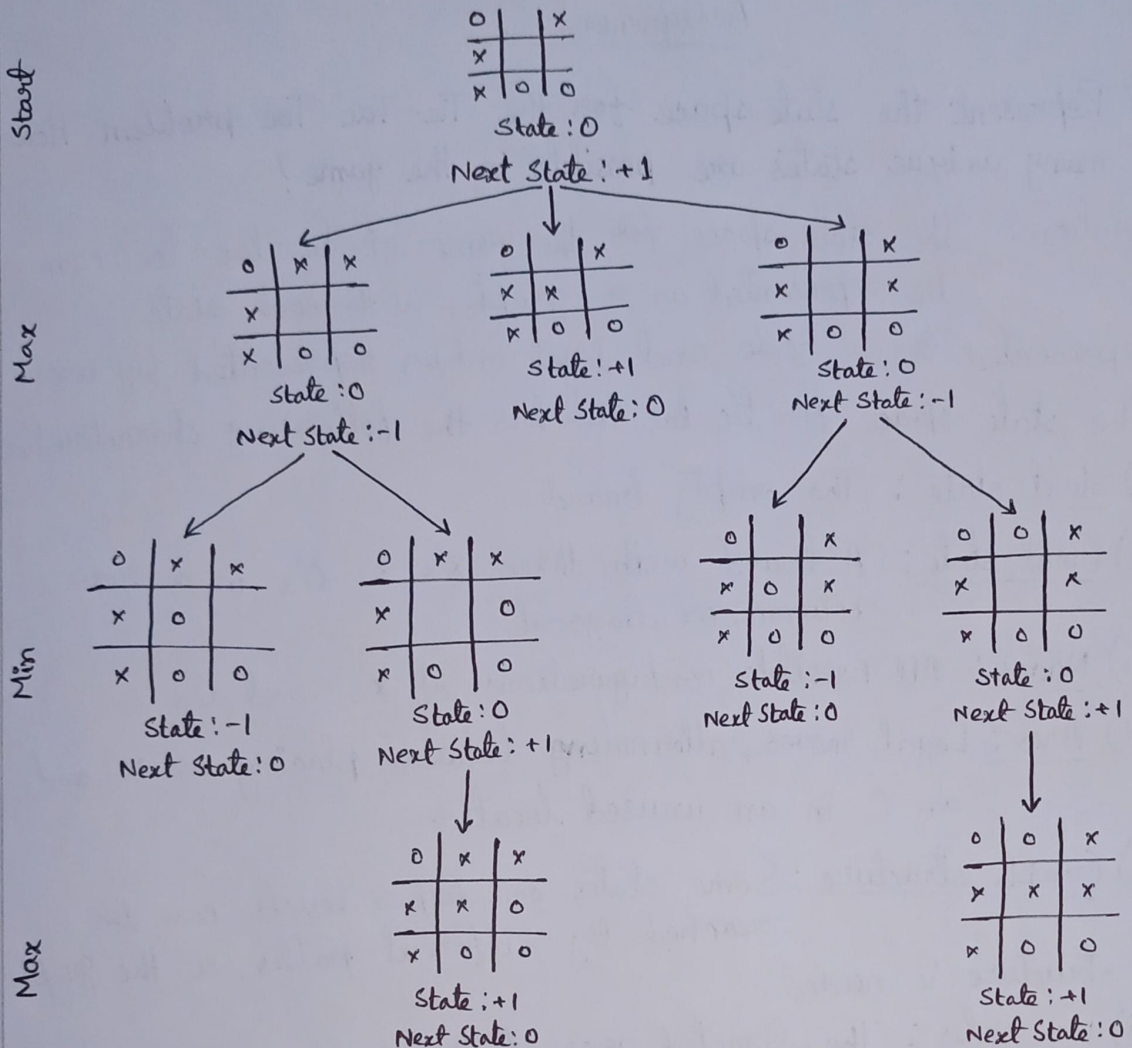


Assignment - 1

1) Represent the state space for the Tic-Tac-Toe problem. How many unique states are possible in the game?

Solution: The state space for the game of tic-tac-toe can be represented as a graph, with each state represented by a node and legal moves represented by arcs. The state space for tic-tac-toe has the following characteristics:

- i) Start state: The empty board
- ii) Goal state: A board with three 'X's or 'O's in a row, column, or diagonal
- iii) Nodes: All possible configurations of X's and O's
- iv) Arcs: Legal moves, alternating between placing an X and an O in an unused location
- v) Graph structure: Some states on deeper levels can be reached by different paths, so the graph structure is needed
- vi) No cycles: The directed arcs of the graph do not allow a move to be undone



The total number of unique states in Tic-Tac-Toe is influenced by several factors, including the number of cells on the board, the valid sequences of moves, and game rules such as wins or draws.

A Tic-Tac-Toe board has 9 cells, and each cell can either:

- Be empty
- Contain an 'X', or
- Contain an 'O'.

Thus, the maximum number of possible board configurations (without considering the rules or symmetries) is:

$$3^9 = 19,683$$

This number includes invalid game states where both players may have made too many moves or a game continued after a win. By taking the valid states and symmetries into account, the number of unique game states reduces significantly. The exact number of unique, valid Tic-Tac-Toe states has been calculated as: 5,478

2) An AI agent for a vacuum cleaner operates in a grid environment. If the grid is 3×3 , how many unique percepts and actions are possible for the agent?

Solution: To determine the number of unique percepts and actions for a vacuum cleaner agent in a 3×3 grid environment, we need to consider both the percepts it can receive from the environment and the actions it can perform.

Percepts —

A percept is the information an agent receives from its environment. In this case, the vacuum cleaner agent can perceive:

(i) Location: The current cell it's in (one of the 9 cells).

(ii) Dirt: Whether there is dirt in the current cell.

So, the number of unique percepts is the combination of these two:

$$9 \text{ possible locations} * 2 \text{ possible dirt states} = 18 \text{ unique percepts}$$

Actions -

The vacuum cleaner agent can perform the following actions:

- (i) Move left
- (ii) Move right
- (iii) Move up
- (iv) Move down
- (v) Suck

Therefore, there are 5 unique actions.

3) Consider an agent that can perceive the temperature of a room and adjust a thermostat. Calculate the number of possible agent-environment interactions if the agent can set the temperature to 5 distinct levels.

Solution: To calculate the number of possible agent-environment interactions, we need to consider two key components:

→ Perceptions: The agent can perceive the temperature of the room.

→ Actions: The agent can set the thermostat to 5 distinct levels.

Since, the temperature can be set to 5 levels, we can assume that there are 5 possible perceptions (e.g., very cold, cold, moderate, warm, very warm).

To calculate the total number of possible agent-environment interactions, we need to multiply the number of possible perceptions by the number of possible actions.

Therefore, the total number of possible agent-environment interactions is :

$$5(\text{perceptions}) * 5(\text{actions}) = 25$$

4) Calculate the number of possible environment states if an autonomous agent operates in a 4-room house, where each room can either be clean or dirty.

Solution: To calculate the number of possible environment states for an autonomous agent operating in a 4-room house, where each room can either be clean or dirty, we can approach the problem as follows:

For each room, there are 2 possible states:

(i) Clean

(ii) Dirty

Since there are 4 rooms and each room's state is independent of the others, the total number of possible environment states is given by:

$$\text{Total Environment States} = 2^{\text{Number of Rooms}} = 2^4 = 16$$

The number of possible environment states for the 4-room house, where each room can be either clean or dirty, is 16.

5) In a dynamic environment, if the agents receive an input every 2 seconds, how many distinct percept-action cycles will occur over 1 hour?

Solution: Convert hours to seconds:

$$1 \text{ hour} = 60 \text{ minutes/hour} * 60 \text{ seconds/minutes} = 3600 \text{ secs.}$$

Divide total seconds by the interval between inputs:

$$3600 \text{ seconds} / 2 \text{ seconds/cycle} = 1800 \text{ cycles.}$$

Therefore, there will be 1800 distinct percept-action cycles over 1 hour.

6) How many different types of agents are there based on their percept history and actions? Give examples and classify each agent type numerically.

Solution: Agents can be classified based on how they use their percept history to determine their actions. Here are the primary types:

- (i) Simple Reflex Agents
- (ii) Model-Based Agents
- (iii) Goal-Based Agents
- (iv) Utility-Based Agents
- (v) Learning Agents

■ (i) Simple Reflex Agents —

→ Behaviour: Acts only based on the current percept.
No use of percept history.

→ Action Rule: Condition-action rules (If-Then rules). The agent selects actions based solely on the present condition (percept).

→ Memory: No memory of previous percepts

Example:

A basic vacuum cleaner agent that sucks dirt if the current room is dirty or moves randomly if the room is

clean.

Numerical Classification:

Based on the current percept only, no memory.

■ Model-Based Reflex Agents —

- Behaviour: Uses a model of the world to keep track of unobservable parts of the environment and updates the state of the environment based on its percept history.
- Action Rule: Uses both the current percept and an internal model of the world.
- Memory: Maintains some internal state representing part of the percept history or environmental model.

Example:

A vacuum cleaner agent that remembers which rooms have been cleaned and chooses actions to visit dirty rooms based on that knowledge.

Numerical Classification:

Based on the current percept plus an internal model.

■ Goal-Based Agents —

- Behaviour: Takes actions that help it achieve a specific goal. The agent makes decisions not only based on the current percept but also by considering future consequences of actions.
- Action Rule: Chooses actions based on goal-directed behaviour. Uses search and planning strategies.

→ Memory: Tracks goal progress and uses the environment's current state.

Example:

A delivery robot that tries to plan a path to deliver a package to a specific destination.

Numerical Classification:

Based on goals, which require searching or planning over percept history.

☐ Utility-Based Agents —

→ Behaviour: Makes decisions based on a utility function that quantifies the desirability of different states. The agent chooses actions that maximize expected utility.

→ Action Rule: Uses a utility function to evaluate and compare the possible outcomes of different actions.

→ Memory: Tracks the percept history and evaluates outcomes for the long-term benefit.

Example:

A self-driving car that decides on the safest, fastest route by evaluating different factors (e.g., traffic, distance) to maximize safety and minimize time.

Numerical Classification:

Based on maximizing utility or preference (more complex than goal-based agents).

Learning Agents —

→ Behaviour:

Learns from past experiences to improve its performance over time. It updates its action strategy based on feedback from the environment.

→ Action Rule: Learns from interaction with the environment to improve its decision-making process. Has a performance element (decision-making) and a learning element (which improves future decisions).

→ Memory: Uses the percept history and stores the outcomes of previous actions to learn better strategies.

Example:

A chess-playing AI that learns from each game to improve its strategies in future games.

Numerical Classification:

Based on learning from past experiences and improving performance.

7) For an AI agent in a real-time strategy game with 5 possible actions and 3 sensor inputs, calculate the number of possible action sequences over 3 time steps.

Solution: To calculate the number of possible action sequences for an AI agent in a real-time-

strategy game over 3 time steps, we need to consider the following key elements:

- Actions: 5 possible actions per time step
- Sensor Inputs: 3 possible sensor inputs per time step
- Time Steps: time steps

Since there are 5 possible actions per time step, the number of possible action sequences for one time step is 5.

For two time steps, it's 5 actions (for the first step) * 5 actions (for the second step) = $5^2 = 25$

Similarly, for three time steps, it's $5^3 = 125$.

Therefore, there are 125 possible action sequences over 3 time steps.

8) For an agent operating in a fully observable, deterministic, and static environment, compute the probability that the agent makes an incorrect decision if its sensors have a 5% error rate.

Solution:

The probability of the sensor giving incorrect data is 5% or 0.05.

Since the environment is deterministic, the agent's decision will only be incorrect if its sensors provide erroneous information. Given a 5% error rate, the probability of the agent receiving a correct sensor reading is $1 - 0.05 = 0.95$ or 95%.

Thus, the probability that the agent makes an incorrect decision is:

$$P(\text{Incorrect Decision}) = P(\text{Sensor Error}) = 0.05$$

The probability that the agent makes an incorrect decision, given that its sensors have a 5% error rate, is 5%.

9) Assume a simple reflex agent can choose from 4 different actions. If it operates in an environment with 10 percepts, calculate the size of the agent function.

Solution:

→ Actions: 4 possible actions

→ Percepts: 10 possible percepts

Since there are 4 possible actions for each percept, the total size of the agent function would be:

$$10 \text{ percepts} * 4 \text{ actions/percept} = 40$$

Therefore, the size of the agent function is 40.

10) Given a problem space of 1000 states, calculate the time complexity of the Breadth-First Search (BFS) algorithm if each state has 4 successors.

Solution:

The time complexity of BFS is generally given by $O(b^d)$, where:

(i) b is the branching factor (number of successors for each state).

(ii) d is the depth of the solution (the number of steps or levels to reach the goal state).

In the worst case, BFS will expand all states before finding the goal. In this case, the time complexity is

proportional to the number of states in the problem space.

(i) Each state has 4 successors (branching factor = 4), but in the worst case, BFS must explore all 1000 states.

Therefore, the time complexity of BFS in this scenario would be:

$$\text{Time Complexity} = O(\text{number of states}) = O(1000)$$

The time complexity of the BFS algorithm for a problem space with 1000 states and a branching factor of 4 is $O(1000)$, because in the worst case, BFS must explore all 1000 states to find the goal state.

11) For a 4-queen problem, if the search space consists of all possible configurations, how many states will be generated during a depth-first search (DFS)?

Solution:

The 4-queen problem involves placing 4 queens on a 4×4 chessboard in such a way that no queen attacks another.

A queen can move horizontally, vertically, and diagonally.

For each queen, there are 4 possible rows to place it. So, for 4 queens, the total number of possible configurations is $4^4 = 256$.

DFS explores a path as deeply as possible before backtracking. In the 4-queen problem, DFS will generate all 256 possible configurations.

Therefore, during a DFS for the 4-queen problem, 256 states will be generated.

12) In a water jug problem where jugs hold 3 and 5 liters, calculate the number of states in the state space if each state represents a unique combination of water in both jugs.

Solution:

In the water jug problem with jugs that hold 3 liters and 5 liters, each state in the state space can be represented as a unique combination of water in both jugs.

→ Jug A (3-liter jug)

→ Jug B (5-liter jug)

(i) For Jug A (3-liter jug), the possible amounts are:

• 0 liters • 1 liter • 2 liters • 3 liters

Total options for Jug A: 4 options

(ii) For Jug B (5-liter jug), the possible amounts are:

• 0 liters • 1 liter • 2 liters • 3 liters • 4 liters
• 5 liters

Total options for Jug B: 6 options.

To find the total number of unique states, multiply the number of options for each jug:

$$\begin{aligned}\text{Total States} &= (\text{Options for Jug A}) \times (\text{Options for Jug B}) \\ &= 4 \times 6 = 24\end{aligned}$$

The total number of unique states in the state space for the water jug problem with a 3-liter jug and a 5-liter jug is 24.

13) For the 8-puzzle problem, calculate the total number of possible states. What is the size of the search space if each tile can move to any adjacent blank position?

Solution:

There are 9 tiles in total, including the blank tile. We can consider the blank tile as a "tile" that can be moved. So, there are $9!$ ways to arrange these 9 "tiles" on the grid.

There are 4 possible rotations for a 3×3 grid.

Therefore, the total number of unique states is:

$$\frac{9!}{4} = 181,440$$

The size of the search space depends on how many states can be generated by moving tiles to adjacent blank positions.

(i) Adjacent Moves: Each tile can move into the adjacent blank space. The maximum number of possible moves at any time can be 2 to 4, depending on the position of the blank space.

(ii) Branching Factor: The branching factor (the average number of possible moves per state) in the 8-puzzle is typically around 2 to 4.

Given these factors, the size of the search space would be determined by the initial state and the number of states reachable from it.

14) Consider a hill-climbing algorithm for the N-queen problem where $N=8$. What is the probability that the algorithm converges to a solution if 30% of the time it gets stuck in local maxima?

Solution:

If 30% of the time the hill-climbing algorithm gets stuck in local maxima, then the probability of it converging to a solution is:

$$1 - 0.3 = 0.7$$

Therefore, the probability that the hill-climbing algorithm converges to a solution for the N-queen problem with $N=8$ is 0.7 or 70%.

15) In adversarial search, calculate the time complexity of the Minimax algorithm for a game with a branching factor of 3 and depth 5.

Solution:

Branching Factor (b) = The number of possible moves from a given state. In this case, $b=3$

Depth (d) = The maximum depth of the game tree. Here, $d=5$

The time complexity of Minimax is $O(b^d)$. This means the number of nodes explored grows exponentially with the depth of the game tree.

For the given problem:

$$\text{Time Complexity} = O(3^5) = O(243)$$

Therefore, the time complexity of the Minimax algorithm for a game with a branching factor of 3 and depth 5 is $O(243)$.

16) If Alpha-Beta Pruning is applied to the Minimax algorithm and the tree is perfectly balanced, calculate the reduction in the number of nodes explored compared to Minimax without pruning.

Solution:

Minimax Without Alpha-Beta Pruning:

The time complexity of Minimax without pruning is $O(b^d)$ where, b is the branching factor (number of possible moves at each node), d is the depth of the game tree.

Minimax With Alpha-Beta Pruning:

The time complexity with Alpha-Beta pruning in this ideal case is $O(b^{d/2})$

Comparison:

The ratio of the number of nodes explored by Minimax without pruning to the number of nodes explored by Minimax with Alpha-Beta pruning is:

$$\text{Reduction Factor} = O(b^d) / O(b^{d/2}) = b^{d/2}$$

This means that the reduction in the number of nodes explored is $b^{d/2}$.

Example Calculation (with $b=3$ and $d=5$):

Let's apply this to an example where the branching factor is 3, and the depth is 5.

(i) Without Alpha-Beta pruning:

$$O(b^d) = O(3^5) = O(243)$$

(ii) With Alpha-Beta pruning:

$$O(b^{d/2}) = O(3^{5/2}) = O(3^2) = O(9)$$

So, in the best-case scenario, Alpha-Beta pruning would reduce the number of nodes explored from 243 to 9.

17) Given a state space of 100 nodes and an evaluation function that takes 0.01 seconds per evaluation, estimate the total runtime for an A* search algorithm if it explores 70% of the nodes.

Solution:

To estimate the total runtime for the A* search algorithm based on the given parameters,

Total Nodes Explored: 70% of 100 nodes = 70 nodes

Evaluation Time per Node: 0.01 seconds

70 nodes * 0.01 seconds/node = 0.7 seconds

Therefore, the estimated total runtime for the A* search algorithm is 0.7 seconds.

18) In propositional logic, compute the number of possible truth assignments for 5 variables.

Solution:

For each variable, there are 2 possible truth values: true or false

For 5 variables, the total number of possible truth assignments is $2^5 = 32$

Therefore, there are 32 possible truth assignments for 5 variables in propositional logic.

19) A propositional logic formula has 4 literals and 3 conjunctions. How many unique interpretations satisfy the formula?

Solution:

To determine how many unique interpretations satisfy a propositional logic formula with 4 literals and 3 conjunctions, we need to consider several factors, including the literals, their truth assignments, and how the conjunctions (AND operations) affect the satisfiability of the formula.

(i) Literals: Propositions or their negations (e.g., p , $\neg q$)

(ii) Conjunctions: Logical AND operations ($\wedge$)

A satisfying interpretation is a truth assignment for the variables that makes the formula true.

Possible Truth Assignments:

Each of the 4 literals can be true or false, so there are $2^4 = 16$ possible truth assignments for the literals.

Conjunction of Literals:

A formula with 3 conjunctions means that the formula takes the form:

$$(L1 \wedge L2 \wedge L3 \wedge L4)$$

where $L1, L2, L3, L4$ are the 4 literals.

For a conjunction formula to be satisfied, each literal must evaluate to true. Out of the 16 possible truth assignments for the 4 literals, only 1 of these assignments

(where all literals are true) will satisfy the formula. There is 1 unique interpretation that satisfies the propositional logic formula with 4 literals and 3 conjunctions.

20) In a Wumpus world scenario, if the agent knows that there are 4 pits in a 5×5 grid, compute the number of possible worlds that satisfy this condition.

Solution:

The Wumpus World is a grid-based environment where an agent must navigate to find gold while avoiding pits and a Wumpus.

Grid Size: $5 \times 5 = 25$ squares

Number of Pits: 4

To calculate the number of possible worlds, we need to find the number of ways to place 4 pits in 25 squares.

This is a combination problem.

$${}^nC_r = \frac{n!}{r! * (n-r)!}$$

where, n = total number of items (squares)

r = number of items to choose (pits)

So, the number of possible worlds is:

$${}^{25}C_4 = \frac{25!}{4! * (25-4)!}$$

$${}^{25}C_4 = 12,650$$

Therefore, there are 12,650 possible worlds that satisfy the condition of having 4 pits in a 5×5 grid Wumpus world.